GUIDE 28 · EVENTS & EXPORTS

Build a client that watches your saved queries run in real time

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=28-build-a-client-that-watches-queries-run. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** a standalone client - Python or browser JavaScript - that receives a live event the moment any saved query - or ad-hoc SQL - runs, with real reconnect-on-drop handling. Both examples below are verified against a real running server, including a real mid-stream server drop and reconnect.

> **Stable** - live events (`GET /events`) are under the compatibility promise since 0.34 (what that means): new event types and fields may be added, so skip one you don't know.

This is about building your *own* consumer of `GET /events` - if you just want to know what the endpoint does and how per-key filtering works, see Give each user of your app their own private activity feed instead; this guide is the "how do I actually parse the stream" one both of those build on.

The wire format, exactly

```text
data: {"type": "query.run", "source": "rest", "occurred_at": "2026-10-13 18:33:58", "kind": "saved",
"query": "example_top_films", "version": 1, "connection_name": "examples", "entry": {"executed_at":
"2026-10-13 18:33:58", "status": "success", "rows": 10, "duration_ms": 4, "key_name": "admin",
"transport": "rest", ...}}

: keepalive
```

Two kinds of frame, each separated by a blank line (`\n\n`):

- **`data: <json>`** - a real event, exactly once per run. `type` is `query.run` for every run; `kind` is `saved` (with `query` and `version`) or `adhoc` (both `null`). Other types will be added - skip one you don't know. Every field is in Live events.
- **`: keepalive`** - a comment line sent every 15 seconds when nothing happened, purely to keep the connection alive through a proxy or client library that would otherwise time out an idle connection. Not JSON, not meant to be parsed - just skip any frame that isn't `data:` .

The one non-obvious thing that will trip you up

The server sends nothing at all - not even the HTTP status line - until the first frame is ready to go. If nothing happens for a while, that means a genuinely idle connection can sit silent for up to 15 seconds (the keepalive interval) before you see *anything*, including confirmation the connection even succeeded. Verified directly: a client with a 3-second read timeout on the initial connection got a spurious timeout error on an idle server, while the identical request with a long (or no) read timeout succeeded cleanly a few seconds later once the first keepalive fired. **Use a long or infinite read timeout on the connection itself** - a short timeout here doesn't mean "the server is down," it means "nothing has happened yet," and those are very different things to react to.

A Python client, with real reconnect-on-drop

```python
import json
import time
import requests

URL = "http://127.0.0.1:5000/events"
KEY = "your-api-key"


def watch(on_event, max_retries=5):
    backoff = 1
    attempt = 0
    while attempt < max_retries:
        try:
            with requests.get(URL, headers={"X-API-Key": KEY}, stream=True, timeout=(5, None)) as res:
                res.raise_for_status()
                print(f"connected (status {res.status_code})")
                attempt = 0  # a successful connect resets backoff
                buffer = ""
                for chunk in res.iter_content(chunk_size=None, decode_unicode=True):
                    if chunk is None:
                        continue
                    buffer += chunk
                    while "\n\n" in buffer:
                        frame, buffer = buffer.split("\n\n", 1)
                        if frame.startswith("data: "):
                            on_event(json.loads(frame[len("data: "):]))
                        # else: a ": keepalive" comment line - ignored
        except requests.RequestException as error:
            attempt += 1
            print(f"stream dropped ({error}); reconnecting in {backoff}s (attempt
{attempt}/{max_retries})")
            time.sleep(backoff)
            backoff = min(backoff * 2, 30)


def print_event(event):
    entry = event["entry"]
    what = f"{event['query']} v{event['version']}" if event["kind"] == "saved" else "ad-hoc SQL"
    print(f"[{event['occurred_at']}] {what} by {entry['key_name']}: "
          f"{entry['status']} ({entry.get('rows', '?')} rows, {entry.get('duration_ms', '?')} ms)")


if __name__ == "__main__":
    watch(print_event)
```

To receive less, ask for it in the URL: `/events?query=example_top_films` for one query's runs, `/events?status=error` for failures only. The filters combine, and work on both the main server and `queryapigate events`.

Note `timeout=(5, None)` - a 5-second *connect* timeout (fine to keep short, TCP connect is fast or it isn't), but no read timeout at all, for exactly the reason above.

Verified for real, including the drop: run this, call a saved query in another terminal, see it print immediately. Then kill the server mid-stream:

```text
connected (status 200)
[2026-10-13 18:33:58] example_top_films v1 by admin: success (10 rows, 4 ms)
stream dropped (Response ended prematurely); reconnecting in 1s (attempt 1/5)
stream dropped (...Connection refused); reconnecting in 2s (attempt 2/5)
stream dropped (...Connection refused); reconnecting in 4s (attempt 3/5)
```

Exponential backoff (1s, 2s, 4s, … capped at 30s), and a successful reconnect resets it back to 1s for the *next* drop - so a flaky connection doesn't end up permanently backed off from one bad moment.

A browser client - the exact approach the admin UI itself uses

`EventSource` can't set the `X-API-Key` header this endpoint needs, so the client reads the stream with `fetch()`'s own streamed response body instead - the same approach the admin UI's Home screen uses (`frontend/src/features/home/HomePage.tsx`), adapted to stand alone:

```javascript
async function watchEvents(apiKey, onEvent) {
  const controller = new AbortController();
  const res = await fetch('/events', { headers: { 'X-API-Key': apiKey }, signal: controller.signal });
  if (!res.ok || !res.body) throw new Error('stream unavailable');

  const reader = res.body.getReader();
  const decoder = new TextDecoder();
  let buffer = '';
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    const parts = buffer.split('\n\n');
    buffer = parts.pop();  // the last (possibly incomplete) frame stays buffered
    for (const part of parts) {
      if (part.indexOf('data: ') !== 0) continue;  // a keepalive comment, or nothing useful
      onEvent(JSON.parse(part.slice(6)));
    }
  }
  return controller;  // call controller.abort() to disconnect deliberately
}
```

The admin UI's own reconnect strategy, worth knowing since it's a real, deliberate design choice and not the only valid one: it does **not** retry the SSE connection itself on a drop - it falls back to 5-second polling instead, and only re-opens the stream the next time Home is opened. That's the right call for a UI panel where "briefly polling instead of streaming" is invisible to the person looking at it; the exponential-backoff reconnect in the Python example above is usually the better choice for a standalone client with no human watching a screen to paper over the gap.

Next steps

- Give each user of your app their own private activity feed - per-key filtering, JWT users, and resuming with `Last-Event-ID`.
- Set up your first scoped API key - use a real scoped key here, not the admin one, once you're past just experimenting.